
PROJECT: MUSIC LISTENER SEGMENTATION

(UNSUPERVISED MACHINE LEARNING)

PROJECT ARCHITECTURE & FILE PURPOSES

1. requirements.txt

- Purpose: Dependency Manifest
- Role: Tells Python which external libraries need to be installed for the project, such as pandas, scikit-learn, joblib and Flask.

2. music_listeners.csv

- Purpose: Dataset
- Role: Contains listener behaviour data used to train the clustering model. The project uses listening hours per week, songs per day, skip rate and playlist count.

3. train_model.py

- Purpose: ML Training Script
- Role: Loads the dataset, selects the features, scales the data using StandardScaler, trains the K-Means clustering model, identifies the listener groups, and saves the trained model and scaler.

4. model.pkl

- Purpose: Saved ML Model
- Role: Stores the trained K-Means model so that the application can assign a new listener to a learned cluster without training the model again.

5. scaler.pkl

- Purpose: Saved Preprocessing Object
- Role: Stores the StandardScaler used during training. The same scaler must be used to transform new user input before prediction.

6. app.py

- Purpose: Backend / Application
- Role: Loads the saved model and scaler, receives listener details, preprocesses the input, predicts the cluster and returns the listener segment.

MODEL FLOW

1. Read music_listeners.csv
2. Select listener behaviour features
3. Scale the features using StandardScaler

4. Create K-Means model with 3 clusters
5. Train the model using fit()
6. Find and understand the three listener groups
7. Save model.pkl and scaler.pkl
8. Receive new listener input
9. Scale the new input using the saved scaler
10. Predict the cluster using model.predict()
11. Display the meaningful listener segment

SETUP & RUN INSTRUCTIONS

STEP 1: CREATE PROJECT FOLDER & OPEN IN VS CODE

1. Create a folder named "music-listener-segmentation".
 2. Open VS Code.
 3. Select File > Open Folder... and choose "music-listener-segmentation".
-

STEP 2: INSTALL PYTHON

1. Make sure Python is installed on your system.
 2. Open VS Code again after installation if Python was newly installed.
 3. Open Terminal > New Terminal.
-

STEP 3: CREATE PROJECT FILES

In the VS Code Explorer panel, create the following files:

1. requirements.txt
2. music_listeners.csv

3. train_model.py

4. app.py

After training, these files should also be created:

5. model.pkl

6. scaler.pkl

STEP 4: INSTALL DEPENDENCIES

Open a terminal in VS Code and run:

```
python -m pip install -r requirements.txt
```

STEP 5: CHECK THE DATASET

1. Open music_listeners.csv.

2. Identify these four features:

- listening_hours_per_week
- songs_per_day
- skip_rate
- playlist_count

3. Notice that there is no target column such as Casual, Explorer or Heavy.

STEP 6: TRAIN THE MODEL

Run:

```
python train_model.py
```

The script should:

1. Load the CSV file.
2. Select the four features.
3. Scale the features.

4. Create K-Means with 3 clusters.
5. Train the model using fit().
6. Identify the cluster centres.
7. Save model.pkl.
8. Save scaler.pkl.

STEP 7: UNDERSTAND THE THREE CLUSTERS

After training, inspect the cluster centres and understand the behaviour of each group.

1. Lower listening activity → Casual Listener.
2. Medium listening activity → Music Explorer.
3. Higher listening activity → Heavy Listener.

Important: K-Means itself produces cluster numbers such as 0, 1 and 2. The meaningful names are assigned after interpreting the cluster centres.

STEP 8: RUN THE APPLICATION

Run:

```
python app.py
```

The application loads model.pkl and scaler.pkl and waits for a new listener's details.

STEP 9: TEST A NEW LISTENER

Enter values for:

1. Listening hours per week
2. Songs per day
3. Skip rate
4. Playlist count

The application then:

1. Receives the values.

2. Converts them into the required format.
3. Uses `scaler.pkl` to scale them.
4. Uses `model.pkl` to predict the cluster.
5. Displays the listener segment.

WHAT YOU ARE USING FROM SUPERVISED LEARNING

This project is Unsupervised Learning. However, you are reusing several ML concepts that were also used in the supervised-learning project.

Features (X): YES — The four listener behaviour columns are the input features.

Target (y): NO — There is no known target/answer column.

Data preprocessing: YES — `StandardScaler` is used before model training.

Model creation: YES — A machine learning model is created.

fit(): YES — The model learns from the training data.

predict(): YES — The trained model assigns a new listener to a cluster.

Train/Test Split: NO — Not used in this simple project.

Classification: NO — The model is not predicting a predefined class.

Regression: NO — The model is not predicting a continuous target.

Accuracy / Precision / Recall: NO — These supervised classification metrics are not used here.

Cross-Validation: NO — Not used in this project.

Hyperparameter Tuning: NO — The project uses fixed K-Means settings rather than searching for the best settings.

Joblib: YES — Used to save and load the trained model and scaler.

STUDENT CHECKLIST

1. Create the project folder.
2. Create the required files.
3. Install the required packages.

4. Open and understand the CSV.
5. Select the four features.
6. Apply StandardScaler.
7. Create K-Means with 3 clusters.
8. Train using fit().
9. Understand the cluster centres.
10. Save model.pkl and scaler.pkl.
11. Run the application.
12. Enter a new listener's details.
13. Scale the new input using the saved scaler.
14. Predict the cluster.
15. Display the listener segment.
16. Explain why this is Unsupervised Learning.

IMPORTANT RULES TO REMEMBER

1. During training, use fit_transform() on the training features.
2. During prediction, use transform() only. Do not fit the scaler again.
3. Use the same feature order during training and prediction.
4. Cluster numbers are not automatically meaningful labels.
5. model.pkl is the trained model; scaler.pkl is the preprocessing object.
6. Train the model before running the application for the first time.

=====